Long-term distribution grid planning with inflated grid models in power-grid-model

Martin Hofmann

martin.hofmann-3@ei.thm.de

0009-0005-7310-8017

About me

SHORT CV

Martin Hofmann, M.Sc.
Electrical Engineering and Information Technology

Distribution Grid Planning and Operation
Research Associate since 2017

THM - University of Applied Sciences, Germany
Research Associate since 2019

Tampere University, Finland
Doctoral student of Computing and Electrical Engineering since 2024

Code and Slides

WHAT I WORK ON

Automated route planning for cables: the open-source tool PYORPS `github.com/marhofmann/pyorps`

N-1-secure planning of medium- and high-voltage grids

High-performance long-term target-grid planning



Long-term planning:
An inflated model and its solution vector

Goal. Plan the distribution grid decades ahead. Load growth, PV, EVs and heat pumps are uncertain, so there are many scenarios.

Measures. Many candidates: Which of them to build is the decision.

Inflated model. All candidates exist in one model, switched off. A solution is a 0/1 vector of fixed length, one bit per measure.

One batch call evaluates thousands of solution vectors. Any optimiser (heuristics, genetic algorithm, local search, LP and others) sits on top.

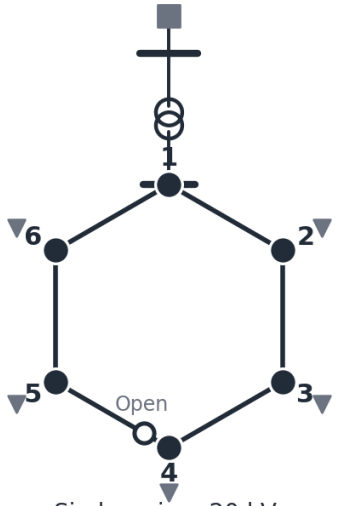
Bit	Measure	Candidate in the model	Bit = 1
1	Parallel line 1-2	Disabled copy of line 1-2	Copy enabled
2	Parallel line 6-1	Disabled copy of line 6-1	Copy enabled
3	Reconductoring 2-3	Disabled line, thicker cable	Original disabled, alternative enabled
4	New line 2-5	Disabled line between buses 2 and 5	Line enabled
5	Open point 5-6 (bound to bit 4)	Disabled -> Closed Line	Line opened
6	Transformer swap	Disabled larger unit	Swap on, original off
7	Transformer parallel	Disabled second unit	Second unit on
8	Move open point	None: existing lines	Line 3-4 opened, Line 4-5 closed

All-off vector: $[0_1\ 0_2\ 0_3\ 0_4\ 0_5\ 0_6\ 0_7\ 0_8]$ = the base grid.
Bit 1 is the first position.

Model Inflation for long term planning

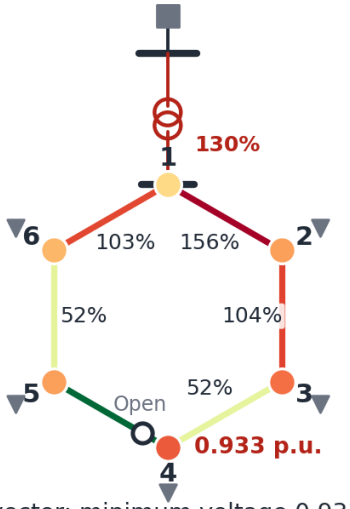
(public pandapower grid)

A Base grid



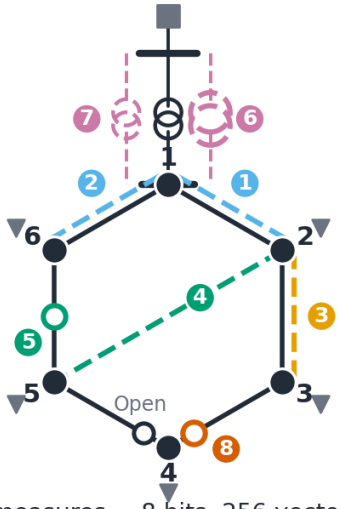
Six-bus ring, 20 kV,
one open point

B Planning scenario: load x6



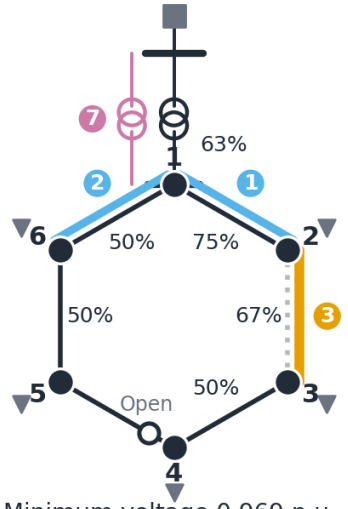
All-off vector: minimum voltage 0.933 p.u.,
loading up to 156 % (limit 80 %)

C Inflated model: candidates (off)



8 measures = 8 bits, 256 vectors,
128 feasible

D Best design 11100010



Minimum voltage 0.969 p.u.,
loading up to 75 %

1 Parallel line 1-2 · 2 Parallel line 6-1 · 3 Reconductoring 2-3 · 4 New line 2-5 · 5 Open point 5-6 (new line) · 6 Transformer swap · 7 Transformer parallel · 8 Move open point

```
solutions = np.array(list(itertools.product([0, 1], repeat=len(MEASURES))))
result = model.calculate_power_flow(
    update_data=status_updates(data, ids, solutions),
    calculation_method=CalculationMethod.newton_raphson,
    continue_on_batch_error=True,
)
```

- **8 measures = 8 bits = 256 vectors, 128 feasible, one batch call.**
- **B – Planning scenario: load x6:**
  Min. voltage = 0.93 p.u.; Max. loading: line = 156 %; Trafo = 130 %
- **D – Best design [1_1 1_2 1_3 0_4 0_5 0_6 1_7 0_8]:**
  Min. voltage 0.97 p.u.; Max. loading: line = 75 %; Trafo = 63 %

Solutions and status updates

```
>>> import itertools
>>> import numpy as np
>>> from power_grid_model import CalculationMethod, PowerGridModel
>>> from small_ring import load_base, inflate, status_updates
>>> data, ids = inflate(load_base())
>>> model = PowerGridModel(data)
>>> MEASURE_TABLE = [
...     ("Parallel line 1-2", 0, 40.0),
...     ("Parallel line 6-1", 0, 40.0),
...     ("Reconductoring 2-3", 1, 60.0),
...     ("New line 2-5", 2, 90.0),
...     ("Open point 5-6 (new line)", 2, 5.0),
...     ("Transformer swap", 3, 120.0),
...     ("Transformer parallel", 3, 100.0),
...     ("Move open point", 4, 5.0),
... ]
>>> solutions = np.array(list(itertools.product([0, 1], repeat=len(MEASURE_TABLE))))
>>> solutions.shape
(256, 8)
>>> solutions[226]
array([1, 1, 1, 0, 0, 0, 1, 0])
>>> upd = status_updates(data, ids, solutions)
>>> upd['line'].shape
(256, 10)
>>> upd['transformer'].shape
(256, 3)
>>> result = model.calculate_power_flow(
...     update_data=status_updates(data, ids, solutions),
...     calculation_method=CalculationMethod.newton_raphson,
...     continue_on_batch_error=True,
... )
>>> result['line'].shape
(256, 10)
```

Bit	Measure	Candidate in the model	Bit = 1
1	Parallel line 1-2	Disabled copy of line 1-2	Copy enabled
2	Parallel line 6-1	Disabled copy of line 6-1	Copy enabled
3	Reconductoring 2-3	Disabled line, thicker cable	Original disabled, alternative enabled
4	New line 2-5	Disabled line between buses 2 and 5	Line enabled
5	Open point 5-6 (bound to bit 4)	Disabled -> Closed Line	Line opened
6	Transformer swap	Disabled larger unit	Swap on, original off
7	Transformer parallel	Disabled second unit	Second unit on
8	Move open point	None: existing lines	Line 3-4 opened, Line 4-5 closed

All-off vector: [0_1 0_2 0_3 0_4 0_5 0_6 0_7 0_8] = the base grid.
Bit 1 is the first position.

The issue: One missing reference

```
template <typename Component, solver_output_type SolverOutputType>
constexpr auto get_component_output(MathOutput<std::vector<SolverOutputType>> const& math_output,
                                    Idx2D const& math_id) {
    auto const& solver_output = math_output.solver_output[math_id.group];
// ...
    auto const& component_type_output = [&solver_output] {                    1
// ...
            return solver_output.load_gen;                                    2
// ...
    }();                                                                      3
// ...
    return component_type_output[math_id.pos];                                4
```

1 The lambda has no return type, so auto is deduced as the container itself.

2 return solver_output.load_gen copies the whole vector:
one heap allocation plus N x 32 bytes.

3 auto const& only extends the life of that copy.

4 One element is read, the copy is destroyed.

Called once per appliance (topological_node_output.hpp:59), for every scenario: N appliances = N copies of N elements -> $O(N^2)$.

8000 loads: 8000 x 8000 x 32 B = about 2 GB copied per scenario.

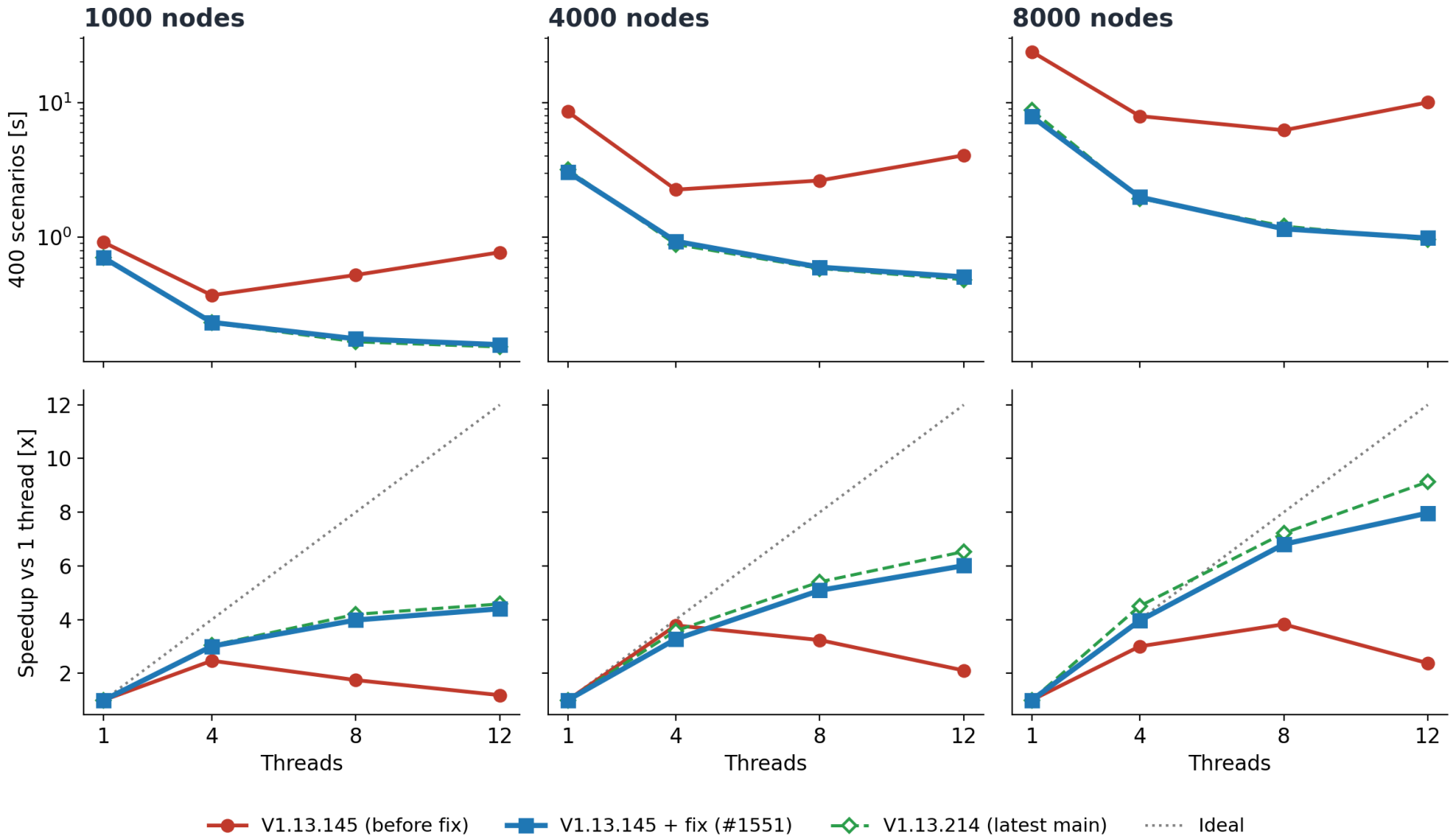
The fix: Say what you mean

```
  template <typename Component, solver_output_type SolverOutputType>
- constexpr auto get_component_output(MathOutput<std::vector<SolverOutputType>> const& math_output,
-                                     Idx2D const& math_id) {
+ constexpr auto const& get_component_output(MathOutput<std::vector<SolverOutputType>> const& math_output,
+                                           Idx2D const& math_id) {
      auto const& solver_output = math_output.solver_output[math_id.group];

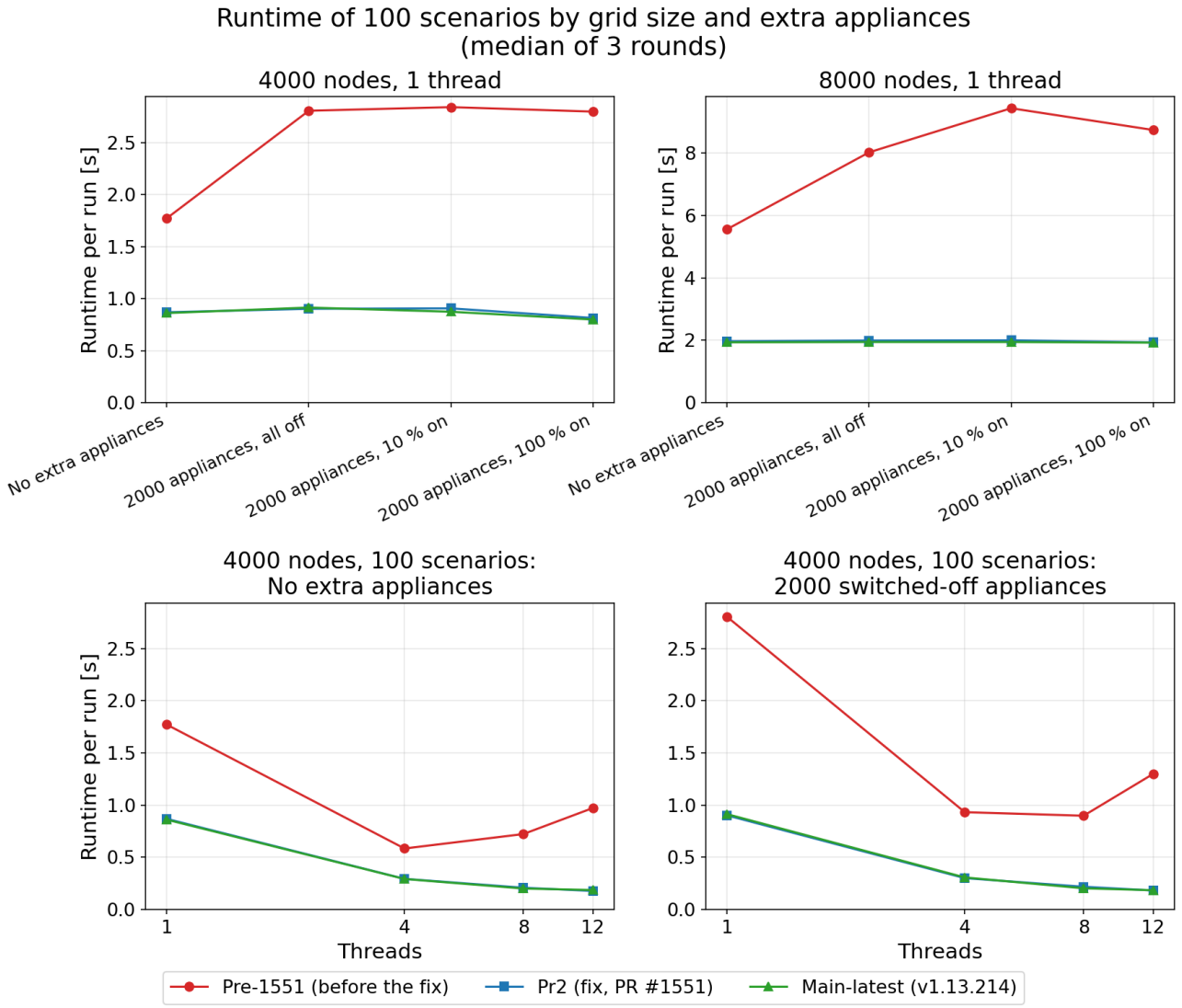
-     auto const& component_type_output = [&solver_output] {
+     auto const& component_type_output = [&solver_output]() -> auto const& {
          if constexpr (std::derived_from<Component, Branch> || std::derived_from<Component, Branch3>) {
              return solver_output.branch;
```

- **-> auto const& on the lambda:** it returns a reference into the container instead of a copy.
- **auto const& on the function:** the reference is passed on to the caller. It points into math_output.solver_output[group], which the caller owns for the whole call.

Before the fix, more cores stopped helping



Switched-off appliances: Costly before the fix, nearly free after



- **Inflated models carry many switched-off appliances.** Before the fix, 2000 of them that never carry load added 50 % to the batch runtime (1.7 s to 2.6 s on 1 thread). With the fix only +7 % (0.79 s to 0.84 s on 1 thread).
- **The cost comes from the number of appliances in the model, not from their status.** Switching them on changes nothing before the fix (2.6 s with 10 % on, 2.5 s with all on).
- Peak memory does not change: 230 MB at 8000 nodes in all builds, about 1.0 GB in the planning workflow. The cost is time.

Takeaways

Tampere University

THM
TECHNISCHE HOCHSCHULE MITTELHESSEN

- **Planning with inflated models means thousands of designs per batch**
  - Small per-value costs add up
- **Test scaling with model size and with thread count early**
  - Small test grids hid this bug
- **Write reference return types explicitly**
  - A deduced return type can silently copy
- Result:
  - about **3.5x faster at 8000 nodes on one thread,**
  - about **6x on 12 threads**,
  - identical results.

Code and Slides

Builds, toolchain, dependencies

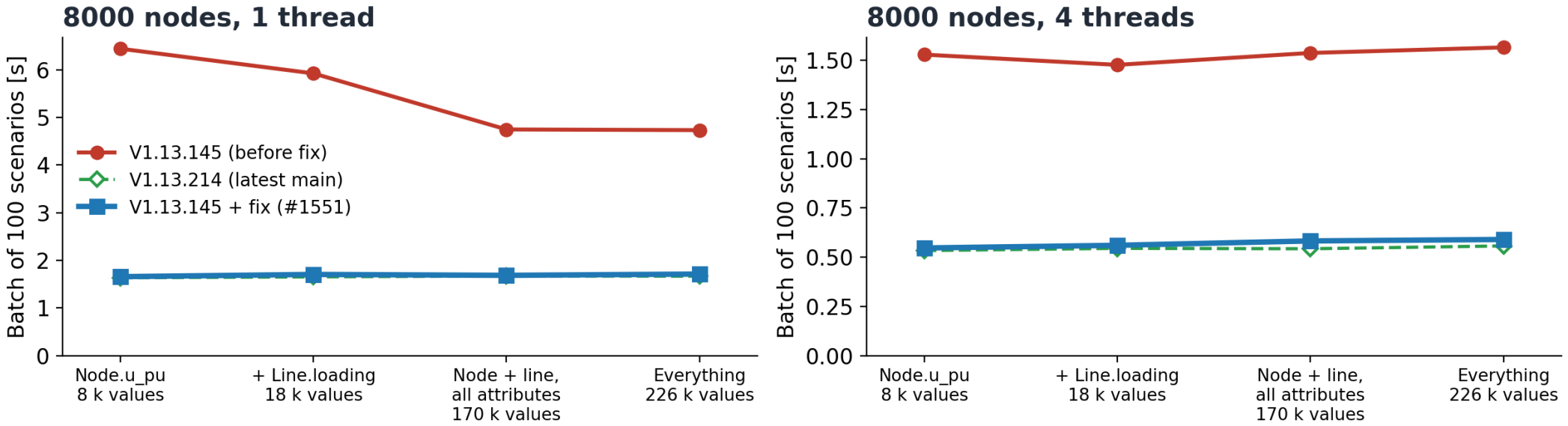
Build	Version	Commit	Note
Before fix	Power-grid-model v1.13.145	3394561ef (2026-08-19)	Parent of the fix
With fix	V1.13.145 + PR #1551	c763f4846 (2026-08-25)	Only the 3-line fix on top
Latest main	V1.13.214	4bd967f57 (2026-10-27)	Adds later PRs (links merged into nodes, #1556, #1555, ...)

Same for all three builds	
Compiler	MSVC 19.51.36260 (toolset 14.51.36231, Visual Studio 18), x64, Release: /O2 /Ob2 /GL /DNDEBUG /EHsc, -std:c++latest
Build tools	CMake 4.3.1, Ninja 1.13.2, pgm-build-deps 2026.8.17.65 (dev build off)
Dependencies	Boost 1.92, Eigen 5.0.1, nlohmann_json 3.12.0, msgpack-cxx 8.0.0
Requirement	No version needed a different dependency version: All three compile and run unchanged with this set.

Run-time stacks	
Synthetic feeder	Python 3.13.5, numpy 2.2.6, psutil 7.2.2; one fresh process per run, builds interleaved, 3 rounds, medians
Planning workflow	Python 3.13.5, numpy 2.2.6, pandas 2.3.2, pandapower 3.4.0, power-grid-model-io 1.3.13
Ring example	Python 3.13.5, pandapower 3.2.2, power-grid-model-io 2.0.20
Machine	Intel Core Ultra 7 265H (16 cores), 31 GB, Windows 11 Enterprise 10.0.26200; at most 12 threads. Idle machine: every run waited for CPU below 10 % over 3 s

Appendix

What you read does not change the cost

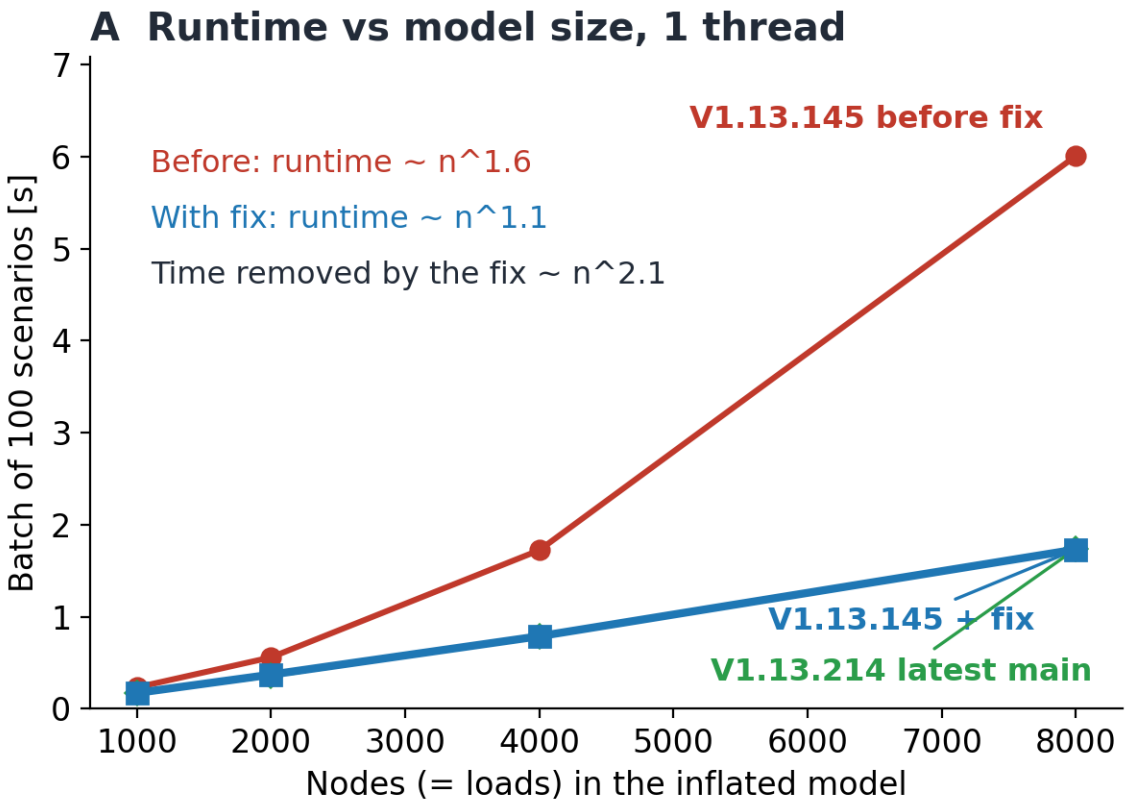


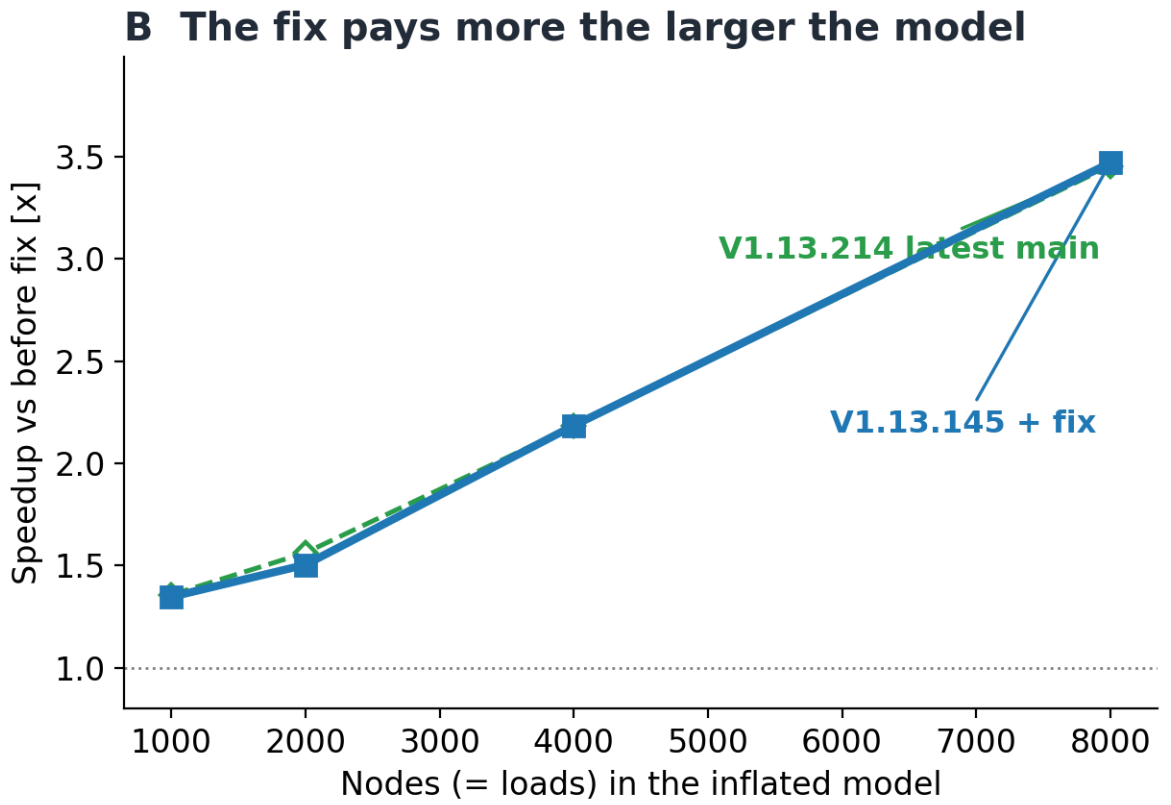
What is read from the solution, per scenario

```
        main_core::solve_topological_nodes(state_, math_output);                    1
// ...
        auto const output_func = [this, &math_output, &result_data]<typename CT>() {
            result_data.for_each_component<typename output_type_getter<SolverOutputType>::type, CT>(
                [this, &math_output](auto const& span) {
                    if (std::empty(span)) {                                         2
                        return;
```

Reading 28x more values per scenario (8 k to 226 k) changes the runtime of the fixed builds by at most 8 % (4000 and 8000 nodes, 1 and 4 threads). The unfixed build varies from round to round regardless of what is read. Mark 1 runs for every scenario; mark 2 only skips writing.

Cost grows with square of appliance count

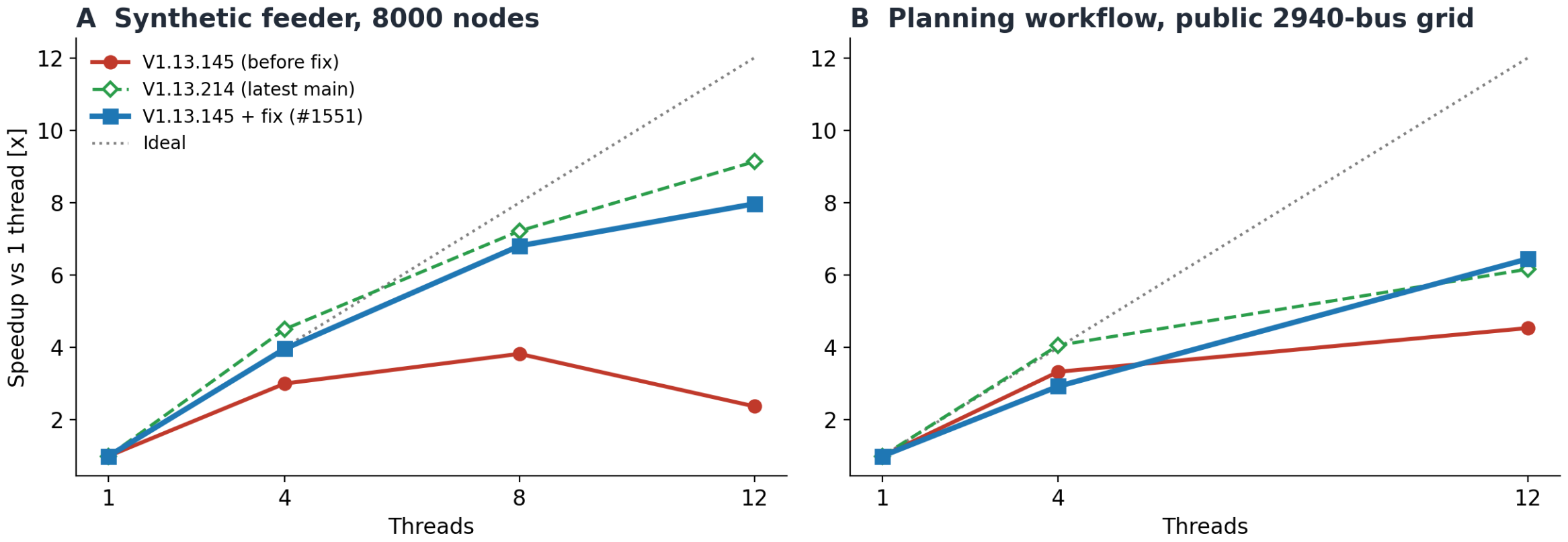




8000 nodes, 1 thread: 6.0 s before the fix, 1.7 s with it (3.5x). The time the fix removes grows as n^2.1: the copies are quadratic. Latest main (v1.13.214) is as fast as the fix alone.

Idle machine. The same configuration gave 2.8x to 3.5x in the three sweeps: the unfixed build varied between rounds (4.6 to 6.6 s in the read sweep); the fixed build stays within 7 %. The cause is not measured.

Before the fix, more threads stopped helping



Before the fix, 8000 nodes: 7.9 s on 4 threads, 6.2 s on 8, 10.0 s on 12 (speedup 3.8x at 8, 2.4x at 12). With the fix: 8.0x on 12 threads. Planning workflow on 12 threads: 1.8 s to 0.9 s (2.0x), identical results. Cause not measured; my hypothesis is allocator and memory-bandwidth contention.